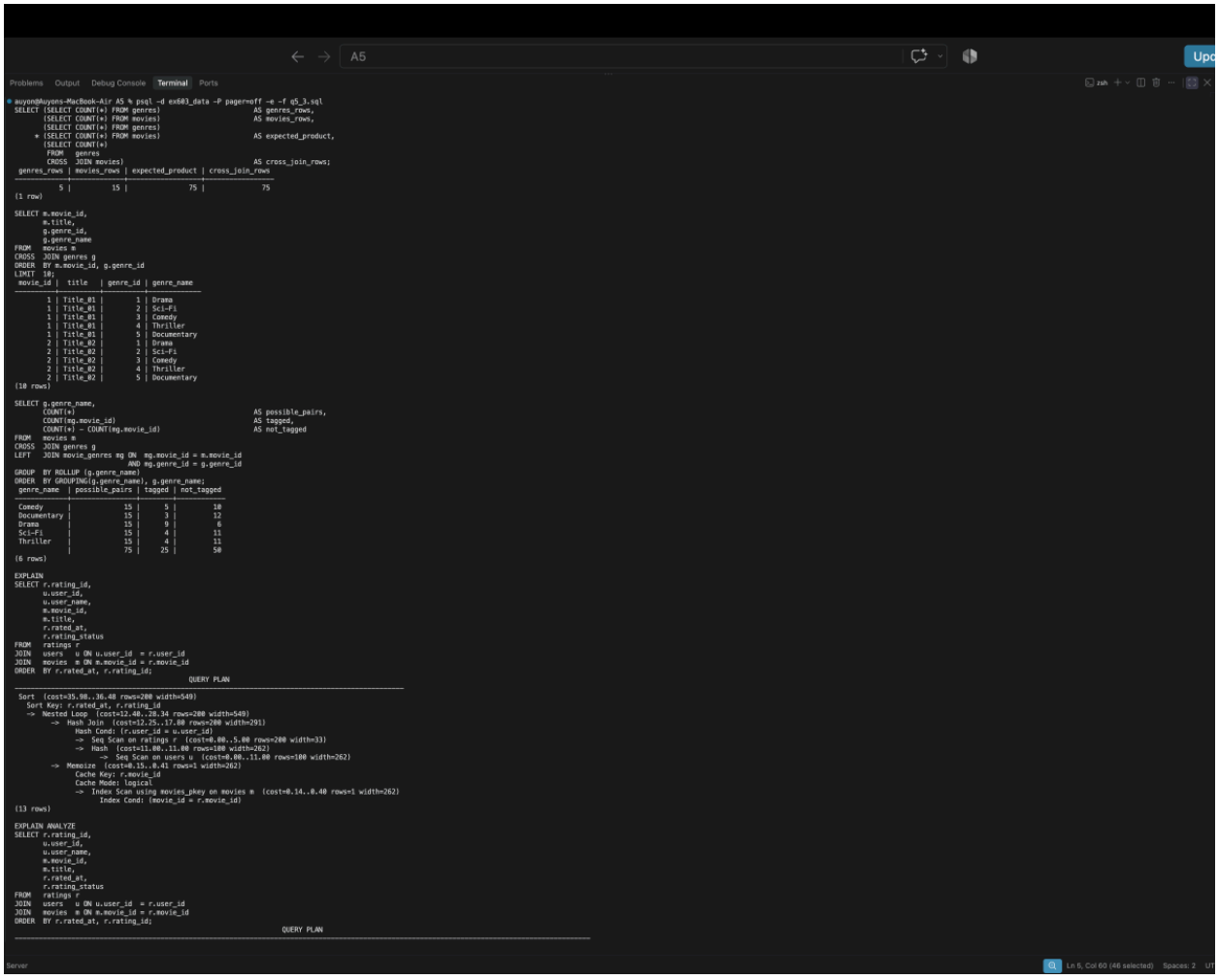

A5
Problems
Output
Debug Console
Terminal
Ports
auyon@auyons-MacBook-Air A5 % psql -d ex603_data -P pager=off -e -f q5_3.sql
SELECT (SELECT COUNT(*) FROM genres)   AS genres_rows,
(SELECT COUNT(*) FROM movies)   AS movies_rows,
(SELECT COUNT(*) FROM genres)
* (SELECT COUNT(*) FROM movies)   AS expected_product,
(SELECT COUNT(*)
FROM genres
CROSS JOIN movies)   AS cross_join_rows;
genres_rows | movies_rows | expected_product | cross_join_rows
5 | 15 | 75 | 75
(1 row)
SELECT m.movie_id,
m.title,
g.genre_id,
g.genre_name
FROM movies m
CROSS JOIN genres g
ORDER BY m.movie_id, g.genre_id
LIMIT 10;
movie_id | title | genre_id | genre_name
1 | Title_01 | 1 | Drama
1 | Title_01 | 2 | Sci-Fi
1 | Title_01 | 3 | Comedy
1 | Title_01 | 4 | Thriller
1 | Title_01 | 5 | Documentary
2 | Title_02 | 1 | Drama
2 | Title_02 | 2 | Sci-Fi
2 | Title_02 | 3 | Comedy
2 | Title_02 | 4 | Thriller
2 | Title_02 | 5 | Documentary
(10 rows)
SELECT g.genre_name,
COUNT(*)   AS possible_pairs,
COUNT(mg.movie_id)   AS tagged,
COUNT(*) - COUNT(mg.movie_id)   AS not_tagged
FROM movies m
CROSS JOIN genres g
LEFT JOIN movie_genres mg ON mg.movie_id = m.movie_id
AND mg.genre_id = g.genre_id
GROUP BY ROLLUP (g.genre_name)
ORDER BY GROUPING(g.genre_name), g.genre_name;
genre_name | possible_pairs | tagged | not_tagged
Comedy | 15 | 5 | 10
Documentary | 15 | 3 | 12
Drama | 15 | 9 | 6
Sci-Fi | 15 | 4 | 11
Thriller | 15 | 4 | 11
| 75 | 25 | 50
(6 rows)
EXPLAIN
SELECT r.rating_id,
u.user_id,
u.user_name,
m.movie_id,
m.title,
r.rated_at,
r.rating_status
FROM ratings r
JOIN users u ON u.user_id = r.user_id
JOIN movies m ON m.movie_id = r.movie_id
ORDER BY r.rated_at, r.rating_id;
QUERY PLAN
Sort (cost=35.98..36.40 rows=200 width=549)
Sort Key: r.rated_at, r.rating_id
-> Nested Loop (cost=12.40..28.34 rows=200 width=549)
-> Hash Join (cost=12.25..17.00 rows=200 width=201)
Hash Cond: (r.user_id = u.user_id)
-> Seq Scan on ratings r (cost=0.00..5.00 rows=200 width=33)
-> Hash (cost=11.00..11.00 rows=100 width=262)
-> Seq Scan on users u (cost=0.00..11.00 rows=100 width=262)
-> Memoize (cost=0.15..0.41 rows=1 width=262)
Cache Key: r.movie_id
Cache Mode: logical
-> Index Scan using movies_pkey on movies m (cost=0.14..0.40 rows=1 width=262)
Index Cond: (movie_id = r.movie_id)
(13 rows)
EXPLAIN ANALYZE
SELECT r.rating_id,
u.user_id,
u.user_name,
m.movie_id,
m.title,
r.rated_at,
r.rating_status
FROM ratings r
JOIN users u ON u.user_id = r.user_id
JOIN movies m ON m.movie_id = r.movie_id
ORDER BY r.rated_at, r.rating_id;
QUERY PLAN
Server
Ln 6, Col 60 (46 selected)
Spaces: 2